DeepLearningCausal: R Package for Estimating Treatment Effects Using Deep Neural Networks and Ensemble Learning

by Nguyen Khoi Huynh, Yang Yang, and Bumba Mukherjee

Abstract Conventional machine learning (ML) methods for causal inference are widely used for estimating causal effects and uncovering heterogeneous treatment effects. Yet, deep learning, which leverages deep neural networks to address selection bias and unobserved confounders, is underutilized for estimating sample and population average treatment effects. Further, conventional ML methods are employed to estimate treatment effects under the assumption that all units in the sample comply with the treatment, which is often implausible. To address these methodological challenges, this article introduces our DeepLearningCausal R-package that enables users to employ deep neural networks to estimate conditional average treatment effects (CATEs) from samples via meta-learners and population average treatment effects on the treated (PATT) in settings with treatment noncompliance. DeepLearningCausal also includes functions to compute and illustrate conformal prediction (CP) intervals for the estimated meta-learner individual treatment effects (ITEs), visualize heterogeneous treatment effects, and estimate CATEs and PATT by using weighted ensemble learning.

1 Introduction

Causal inference constitutes a vital component of scientific research in biology, economics, epidemiology, medicine, political science, and sociology (Pearl, 2009; Imbens and Rubin, 2015; Athey and Imbens, 2016; Li et al., 2024). Many puzzles in the natural and social sciences are causal questions and answers to these questions are statistically evaluated by estimating conditional average treatment effects (CATE) obtained from randomized control trials or (structural) causal models estimated on experimental and observational samples (Hill, 2011; Yao et al., 2018; Hu et al., 2021; Tikka, 2023). Yet the following methodological challenges often impede the accuracy of treatment effects estimated from samples: unobserved confounders, high-dimensional data, low external validity to the target population, and complex non-linear relationships between the treatment and outcome measures.

To address these challenges, scholars use conventional machine learning (ML) algorithms such as causal trees (Athey and Imbens, 2016), kernel methods (Alaa and Van Der Schaar, 2018), or causal boosting (Powers et al., 2018) that capture complex non-linear relationships when estimating treatment effects. Others have used targeted learning which leverages the Super Learner approach for estimating targeted causal estimands (van der Laan and Rose, 2011; Schuler and Rose, 2019). Researchers have also accounted for unobserved confounders by developing *meta-learner* models that tackle the said issue by decomposing estimation of CATEs into multiple sub-problems solved by conventional ML methods (Künzel et al., 2019; Nie and Wager, 2021). Some scholars have addressed external validity by extending the generalizability of sample treatment effects to the target population by using standard ML methods to estimate population average treatment effects on the treated from experimental and observational data (Hartman et al., 2015; Balzer et al., 2016; Zhang et al., 2019; Ottoboni and Populos, 2020). These statistical learning approaches that employ machine learning for causal inference have been widely disseminated via software packages that are discussed in the next section.

But unlike the widespread use of standard ML algorithms for causal inference, deep learning methods have been underutilized for the estimation of not just sample treatment effects such as CATEs but also population average treatment effects. This is surprising as deep learning methods based on deep neural networks capture heterogeneous treatment effects with low bias, account for unobserved confounders and selection bias, and assess causality in complex settings (Farrell et al., 2021; Koch et al., 2024). Moreover, the aforementioned statistical learning approaches are applied to settings which assume that all individual units in the experimental and observational sample have fully complied with the assigned treatment, even though some units (in reality) neither comply with nor accept the treatment. Not addressing treatment noncompliance of this sort leads to biased results when generalizing estimated treatment effects from samples to the broader target population.

We thus present our R package **DeepLearningCausal** (available at `https://cran.r-project.org/package=DeepLearningCausal`) in this paper which overcomes these limitations and enables researchers to employ deep learning methods to estimate CATEs from samples and population average treatment effects on the treated (PATT) from data that exhibit treatment noncompliance. **DeepLearningCausal** allows users to employ deep learning based on deep neural networks to estimate CATEs from four meta-learner models: T-learner, S-learner, X-learner, and R-learner. Our package also pro-

vides functions to use deep neural networks to estimate the PATT from a newly developed causal model that addresses treatment noncompliance by some units in the sample. Further, **DeepLearningCausal** provides an interface to Python that enables users to leverage Python's extensive deep learning ecosystem (within their R script) for estimating the CATE and PATT via deep neural networks. The package provides users with substantial flexibility to substantially customize their deep neural network architecture for deep learning estimation, and enables users to conduct conformal inference of individual treatment effects obtained from the meta-learner models. The analysis presented below primarily focuses on our package's functionality for *deep learning estimation* of the meta-learner CATEs and the PATT (given treatment noncompliance) using **reticulate**(Ushey et al., 2025), **tensorflow** (Allaire and Tang, 2024; Abadi et al., 2015), and **keras3** (Kalinowski et al., 2025; Chollet et al., 2015) which provide access to Python's deep learning libraries. The functions and procedure for estimating the meta-learner CATEs and PATT using weighted ensemble learning and R neural nets are provided in our package's GitHub repository: hknd23/DeepLearningCausal.

2 Background and other R packages

Although parametric models are often used for estimating average treatment effects, they rely on the assumption that the functional form of the relationship between variables is correctly specified. But if the assumed model is misspecified, the estimates can be biased, and identification of causal effects can be challenging in the presence of unmeasured confounders (Jonzon et al., 2023; Tikka, 2023). Scholars have thus used conventional machine learning methods (e.g., gradient boosting, random forests) to address the aforementioned challenge when estimating CATEs in samples or the PATT in the target population, and one recent study uses deep learning for estimating CATEs from two meta-learner models (Hartman et al., 2015; Johansson et al., 2016; Chernozhukov et al., 2018; Yao et al., 2018; Künzel et al., 2019; Zhang et al., 2020; Koch et al., 2024). Deep learning techniques for causal inference that address challenges such as unstructured variables, unobserved confounders, or unknown intervention include either the use of *general* deep neural networks that involve using multilayer perceptrons or *specific* deep neural network architectures including convolutional neural networks and variational autoencoders (Goodfellow et al., 2016; Koch et al., 2024). Other studies have implemented "targeted learning" that employs super learner-based weighted ensemble learning to estimate specific causal parameters (estimands) by *targeting* the data-fitting process to the target parameter (van der Laan and Rose, 2011; Schuler and Rose, 2019).

Conventional ML, targeted learning, and deep learning methods for estimating causal effects address the limitations of parametric models as these methods learn complex relationships without requiring explicit functional forms for the association between variables (Zhang et al., 2020; Nie and Wager, 2021; Okasa, 2022; Hu and Ji, 2022). Given these advantages, it is not surprising that conventional ML or super learner methods are employed for estimating various causal models, including Augmented Inverse Probability Weighting, Double/Debiased Machine Learning, causal estimands that employ targeted learning, and meta-learner models (Glynn and Quinn, 2010; van der Laan and Rose, 2011; Chernozhukov et al., 2018; Knaus, 2022; Bach et al., 2024; Okasa, 2022). While the review by Koch et al. (2024) introduces deep learning for causal effect estimation in the social sciences, researchers also use standard ML methods to estimate population average treatment effects on the treated from experimental data with and without treatment noncompliance (Hartman et al., 2015; Balzer et al., 2016; Ottoboni and Populos, 2020).

In line with these trends in the causal machine learning literature, numerous R packages have been developed that permit users to use machine learning methods for causal inference. The package **DoubleML** Bach et al. (2024) focuses on estimation of the nuisance parts in causal models by machine learning methods and computation of the Neyman orthogonal score functions, while **lmtp** by Díaz et al. (2021) employs ensemble learning for estimating causal effects based on longitudinal treatment indicators. The **CIMTx** package by Hu and Ji (2022) uses Bayesian additive regression trees to estimate multiple treatments with a focus on binary outcomes, and the package **causaloptim** Jonzon et al. (2023) focuses on causal graphs and bounds for causal inference that have implications for machine learning model development. The **SVMMatch** package by Ratkovic (2015) uses support vector machines for causal effect estimation, the **htetree** package by Xu et al. (2023) provides functions to estimate heterogeneous treatment effects with tree-based machine learning algorithms, and **twangContinuous** by Coffman and Griffin (2021) uses gradient boosting machines for estimating continuous treatments. Python packages such as *CausalML* by Zhao and Liu (2023) and *EconML* by Battocchi et al. (2019) also provide ML methods for estimating treatment effects. There also exists open-source R code that uses ML methods for estimating meta-learner models (Künzel et al., 2019; Nie and Wager, 2021).

To our knowledge, our **DeepLearningCausal** package is the first that allows users to employ deep learning based on deep neural networks to estimate CATEs from several meta-learner models *and* the PATT from a new estimator that accounts for treatment noncompliance by some units (Ottoboni

and Populos, 2020). **DeepLearningCausal** also permits users to automatically install the **reticulate** R package, **TensorFlow**, and **Keras3** (R and Python packages) to integrate Python code and libraries used for deep neural networks within an R environment. This feature enables estimation of treatment effects via a highly customized deep neural network architecture using cutting-edge optimizers such as Adam and RMSprop. Further, our package permits users to implement the recently developed conformal prediction (CP) framework to conduct inference of target parameters such as individual treatment effects from meta-learner models. Additional functions in our package permit estimation of the CATE and PATT via the Super Learner-based weighted ensemble learning, illustration of heterogeneous treatment effects, and assessment of correlation of CATEs from meta-learner models.

3 Models

We use the Neyman-Rubin potential outcomes framework to conceptualize estimation of the CATEs and the PATT (Imbens and Rubin, 2015). Assume the dataset $\mathcal{D}(Y_i, \mathbf{X}_i, W_i)_{i=1}^n$ with $(Y_i, \mathbf{X}_i, W_i) \overset{i.i.d}{\backsim} \mathcal{P}$ where i denotes an individual subject, $Y \in \mathcal{Y}$ is a binary or continuous outcome of interest, $\mathbf{X}_i \in \mathcal{X} \subset \mathbb{R}^d$ is the vector of covariates, and $W_i \in \{0,1\}$ is the binary treatment assigned according to the propensity score $\pi(\mathbf{x}) = P(W_i = 1|\mathbf{X}_i = \mathbf{x})$. The two potential outcomes (POs) represent the outcome under the treatment $Y_i^{(1)}$ and control $Y_i^{(0)}$. Since only one of the POs is observed, the observed outcome is: $Y = W_i Y_i^{(1)} + (1 - W_i) Y_i^{(0)}$. The ATE is the difference between the two POs: $\tau = E\left[Y_i^{(1)} - Y_i^{(0)}\right]$. The ATE is *identifiable* when the following assumptions hold: consistency, SUTVA (Stable Unit Treatment Value Assumption), ignorability (POs are independent of the treatment assignment given the covariates), and positivity (each i has a non-zero probability of belonging to the control and treated groups). These assumptions also permit identification of the CATEs defined below.

3.1 CATEs and non-parametric regression framework

Given *identifiability*, the conditional average treatment effect (CATE) is the expected difference between the two POs conditional on the covariates $\mathbf{X}_i = \mathbf{x}$:

$$\tau(\mathbf{x}) = \mathbb{E}\left[Y_i^{(1)} - Y_i^{(0)}|\mathbf{X}_i = \mathbf{x}\right] = \mu_1(\mathbf{x}) - \mu_0(\mathbf{x}) \tag{1}$$

Equation (1) reveals that the CATE is defined as the difference between the response surface under treatment, $\mu_1(\mathbf{x})$, and under control, $\mu_0(\mathbf{x})$:

$$\mu_w(\mathbf{x}) := \mathbb{E}\left[Y^{(w)}|\mathbf{X}_i = \mathbf{x}\right] \tag{2}$$

While the CATE can be estimated via numerous methods, we focus below on *meta-learners* which are non-parametric regression approaches that model the outcome surface Y as a function of the treatment assignment W_i, the covariates $\mathbf{X}_i$, and the error term ε_i: $Y = f(\mathbf{X}_i, W_i) + \varepsilon_i$ where $\varepsilon_i \sim N\left(0, \sigma^2\right)$ and $f(\mathbf{X}_i, W_i) = \mathbb{E}[Y|\mathbf{X}_i, W_i]$. We turn to describe the four meta-learner models that estimate the CATE via the non-parametric regression framework: T-learner, S-learner, X-learner, and R-learner.

3.2 Meta-learner models: T-learner and S-learner

Meta-learner models decompose estimation of the CATE $\hat{\tau}(\mathbf{x})$ into multiple prediction problems (Künzel et al., 2019). The machine or deep learning method employed to solve the prediction problem is called a *base-learner*. Most of the prediction problems in meta-learner models amount to estimating the conditional means of the outcome and the treatment. The latter are referred to as *nuisance* functions, because they are not of primary interest themselves, but are needed to derive $\hat{\tau}(\mathbf{x})$. Meta-learner models are typically categorized into *conditional mean regression* and *pseudo-outcome* methods (Okasa, 2022). We thus formally describe two main conditional mean regression methods (T and S-learner models) and then turn to present two key pseudo-outcome methods (X and R-learner models).

To begin with, the T-learner model estimates $\tau(\mathbf{x})$ by estimating the conditional mean function in each treatment arm, $\mu_0(\mathbf{x}) = \mathbb{E}[Y|\mathbf{X}_i = \mathbf{x}, W_i = 0]$, and the conditional mean function under treatment, $\mu_1(\mathbf{x}) = \mathbb{E}[Y|\mathbf{X}_i = \mathbf{x}, W_i = 1]$. Accordingly, the T-learner estimates the CATE by fitting two separate response surfaces for the treated and control groups respectively, and then computing their difference:

$$\widehat{\tau}(\mathbf{x}) = \widehat{\mu}_1(\mathbf{x}) - \widehat{\mu}_0(\mathbf{x}) \tag{3}$$

This task is implemented by fitting two separate prediction models for the base learner: the first to

the control group's data and the second to the treatment group's data. The base learner can be fitted by weighted ensemble learning or deep neural networks which we focus on below. Specifically, deep neural network (DNN-)estimated CATEs from the T-learner model are obtained by using feed-forward networks. This entails fitting a separate network for each regression task, namely the PO surface for the treated, μ_1, and the control, μ_0, groups. The two networks are trained using,

$$\mathcal{L}_F + \lambda \sum_{1}^{n_w} \Re\left(\Theta_{\mu_W}\right) \tag{4}$$

$\mathcal{L}_F$ is the loss function of the general penalized objective function, $\Re\left(.\right)$ is the regularization term, Θ_{μ_W} is the network's weights where $W_i \in (0,1)$, and λ is the hyperparameter that controls the trade-off between the standard loss and regularization term. Once the two POs surfaces are estimated, CATEs from the DNN-estimated T-learner are obtained by the difference: $\widehat{\tau}\left(\mathbf{x}\right) = \widehat{\mu}_1\left(\mathbf{x}\right) - \widehat{\mu}_0\left(\mathbf{x}\right)$. This means that DNN-estimation of the T-learner fits the response variable Y by assuming that the response surfaces μ_w are group-specific, and thus dependent on different conditional means $f_w\left(.\right)$ and error terms ε_w. The pseudocode for the DNN-estimated T-learner is:

Algorithm: DNN-estimated T-learner	
Input: X, Y, W; **Output:** $\widehat{\tau}$	
1:$\widehat{\mu}_0 = NN_1\left(Y^0 \sim \mathbf{X}^0\right)$	▷ Estimate POs surfaces
2:$\widehat{\mu}_1 = NN_2\left(Y^1 \sim \mathbf{X}^1\right)$	
3: $\widehat{\tau}\left(\mathbf{x}\right) = \widehat{\mu}_1\left(\mathbf{x}\right) - \widehat{\mu}_0\left(\mathbf{x}\right)$	▷ Estimate CATE

where NN_1 denotes deep neural networks. The S-learner uses the whole sample to fit a single model in which the observed outcome values are modeled as a function of the covariates and the treatment to obtain $\widehat{\mu}\left(\mathbf{x}, w\right) = \hat{\mathbb{E}}\left[Y|\mathbf{X}_i = \mathbf{x}; W_i = w\right]$. Hence, the S-learner's CATEs are estimated as,

$$\widehat{\tau}\left(\mathbf{x}\right) = \widehat{\mu}\left(\mathbf{x}, 1\right) - \widehat{\mu}\left(\mathbf{x}, 0\right) \tag{5}$$

This expression implies that the S-Learner fits a single response surface $\mu\left(.\right)$ by using regressors $\left(\mathbf{X}_i, W_i\right)$ through a base learner and estimates CATEs by taking the difference between the two conditional average POs represented by the fitted $\widehat{\mu}\left(.\right)$ with $W_i = 1$ and $W_i = 0$. The group-specific conditional average POs in the S-learner stem from the same model, with conditional mean function $\mu\left(.\right)$ and error term ε_i. Formally, the deep neural network-based (DNN-based) S-learner uses feed-forward networks to fit a separate network for the regression task, namely the single response surface. This network is trained by using the general loss function defined above. Once the response surface is estimated, the DNN-based S-learner obtains the CATE by the difference stated in equation (5). The pseudocode for estimating the DNN-based S-learner is:

Algorithm: DNN-estimated S-learner	
Input: X, Y, W; **Output:** $\widehat{\tau}$	
1: $\widehat{\mu} = NN_1\left(Y \sim (X, W)\right)$	▷ Estimate response surface
2: $\widehat{\tau}\left(\mathbf{x}\right) = \widehat{\mu}\left(x, 1\right) - \widehat{\mu}\left(x, 0\right)$	▷ Estimate CATE

3.3 Individual Treatment Effects (ITEs) and conformal prediction

Existing studies have primarily focused on machine learning algorithms or (less frequently) deep learning to produce point estimates of CATEs from meta-learner models, which quantify the expected difference in treatment outcomes for individuals with specific characteristics (Wager and Athey, 2019; Künzel et al., 2019). While CATEs account for different covariates by averaging effects across similar individuals, it tends to overlook individual-level heterogeneity. Recent studies have addressed this issue by focusing on individual treatment effects (ITEs) that offer more accurate predictions tailored to each individual (Alaa and Van Der Schaar, 2018).

Building on this, our package permits users to apply the recently developed work on weighted split conformal quantile regression (CQR) for producing conformal prediction (CP) intervals for ITEs obtained from the said meta-learner models (Lei and Candès, 2021; Alaa et al., 2023). The CQR framework that integrates the concept of conformal prediction with quantile regression enables direct inference of the target parameters (ITEs) obtained in the two conditional mean regression methods: the S-learner and T-learner model.

Weighted conformal prediction provides a model-agnostic and distribution-free framework that produces interval predictions with the desired coverage probability. The main idea of (split) conformal

prediction is to compute a conformity score by fitting a predictive model on the training set and then evaluating the conformity score on the calibration set to quantify the uncertainty of future predictions. Doing so allows researchers to estimate and construct intervals of ITEs by computing the empirical quantile of conformity scores evaluated in a held-out calibration set. We turn to briefly present below how we employ the weighted split CQR framework to estimate and construct CP intervals for the S-learner model's ITEs. The CQR framework for estimating and constructing CP intervals for the T-learner's ITEs is summarized in the GitHub repository of our package.

To this end, recall that the meta-learner CATEs are defined as $\tau(\mathbf{x}) = \mathbb{E}\left[Y_i^{(1)} - Y_i^{(0)} | \mathbf{X}_i = \mathbf{x}\right]$. Our focus is to employ the weighted split CQR procedure to estimate and construct valid prediction intervals $\widehat{C}_{1-\alpha}(x_i)$ for the S-learner's ITEs: $\widehat{\tau}(x_i) = \widehat{Y}_i(1) - \widehat{Y}_i(0)$ where $\widehat{Y}_i(1)$ and $\widehat{Y}_i(0)$ are predicted potential outcomes under treatment and control, respectively. The algorithm for doing so is summarized as:

Algorithm: Weighted split CQR procedure for S-learner's ITEs

Step 1. Randomly split training data into two parts:
▷ A model training subset to fit the predictive model $\widehat{f}(x, w)$
▷ A calibration subset to compute nonconformity scores and determine the empirical error distribution. Denote the calibration subset as $\mathcal{C}$.
Step 2. Compute Nonconformity Scores:
▷ For each $i \in \mathcal{C}$, compute the absolute residual (nonconformity score) as $r_i = \left|y_i - \widehat{Y}_i(w_i)\right|$. $\widehat{Y}_i(w_i)$ is the model's predicted outcome given treatment w_i
Step 3. Assign Propensity-Based Weights:
▷ Since some calibration units are more informative than others, assign *overlap weights* $g_i = \widehat{p}_i(1 - \widehat{p}_i)$
▷ These overlap weights (which emphasize units in the regions of covariate overlap) are based on the estimated propensity score $\widehat{p}_i = P(W_i = 1 \mid X_i)$
▷ Normalize the weights: $\tilde{g}_i = \frac{g_i}{\sum_{j \in \mathcal{C}} g_j}$
Step 4. Compute the weighted quantile of calibration residuals:
▷ Compute the weighted $(1 - \alpha)$ quantile of the calibration residuals: $q_{1-\alpha} = \min\left\{q : \sum_{i:r_i \leq} \tilde{w}_i \geq 1 - \alpha\right\}$
▷ $q_{1-\alpha}$ represents the smallest residual threshold
Step 5. Construct conformal prediction intervals:
▷ Construct a two-sided conformal interval around the estimated treatment effect for each x_i: $\widehat{C}_{1-\alpha}(x_i) = [\widehat{\tau}(x_i) - q_{1-\alpha},\ \widehat{\tau}(x_i) + q_{1-\alpha}]$

3.4 X-learner and R-learner (meta-learner) models

Pseudo-outcome methods such as the X-learner and R-learner models presented here first estimate nuisance functions (e.g., the conditional means of the outcome and the propensity score) and then combine these estimates into a pseudo-outcome $\hat{\psi}$ (Künzel et al., 2019; Nie and Wager, 2021). The X-learner specifically computes two group-specific estimators, $\hat{\tau}_1(\mathbf{x})$ and $\hat{\tau}_0(\mathbf{x})$, and combines them with a weighting function to estimate the CATEs, $\hat{\tau}(\mathbf{x})$. Künzel et al. (2019) proposed three steps to estimate $\hat{\tau}(\mathbf{x})$ from the X-learner that we adopt. First, we estimate the two PO response surfaces, $\mu_1(\mathbf{x})$ and $\mu_0(\mathbf{x})$, by using deep neural networks (or weighted ensemble learning) as the base-learner. The estimated functions are $\widehat{\mu}_1(\mathbf{x})$ and $\widehat{\mu}_0(\mathbf{x})$. Second, we obtain the imputed treatment effects $\tilde{D}$ by computing the difference between the observed outcomes Y and the counterfactual outcomes estimated for the treatment not assigned using the corresponding regression surface $\widehat{\mu}_w$,

$$\tilde{D}^1 = Y^1 - \widehat{\mu}_0(\mathbf{X}_i) \text{ if } W_i = 1 \tag{6}$$

$$\tilde{D}^0 = \widehat{\mu}_0(\mathbf{X}_i) - Y^0 \text{ if } W_i = 0 \tag{7}$$

for the treated and control groups. If $\widehat{\mu}_0 = \mu_0$ and $\widehat{\mu}_1 = \mu_1$ then $\tau(\mathbf{x}) = \mathbb{E}\left[\tilde{D}_1 | \mathbf{X}_i = \mathbf{x}\right] = \mathbb{E}\left[\tilde{D}_0 | \mathbf{X}_i = \mathbf{x}\right]$. $\tilde{D}$ is an unbiased estimator of τ when μ_0 and μ_1 are known. We next estimate the group-specific CATEs, $\widehat{\tau}_0$ and $\widehat{\tau}_1$, in two separate non-parametric pseudo-outcome regressions, using the imputed treatment effects as the response variable and the covariates $\mathbf{X}$ as regressors:

$$\tilde{D}_1 = \tau_1(\mathbf{X}_i) + \eta_1 \text{ if } W_i = 1 \tag{8}$$

$$\tilde{D}_0 = \tau_0(\mathbf{X}_i) + \eta_0 \text{ if } W_i = 0 \tag{9}$$

η_w is a general error term. $\widehat{\tau}_0$ and $\widehat{\tau}_1$ can be estimated from these expressions by using ensemble learning or deep neural networks as the base-learner. Third, we estimate CATEs by the weighted

average:

$$\hat{\tau}(\mathbf{x}) = g(\mathbf{x})\,\hat{\tau}_0(\mathbf{x}) + (1 - g(\mathbf{x}))\,\hat{\tau}_1(\mathbf{x}) \tag{10}$$

We use the propensity score $g(\mathbf{x}) = \pi(\mathbf{x})$ as the weighting function. This requires estimating $\hat{\pi}(\mathbf{x})$ with a separate neural network (or via ensemble learning). Hence, the pseudocode for obtaining CATEs from the DNN-estimated X-learner is:

Algorithm: DNN-estimated X-learner	
Input: X,Y,W, g; **Output:** $\widehat{\tau}$	
1:$\widehat{\mu}_0 = NN_1\left(Y^0 \sim \mathbf{X}^0\right)$	▷ Estimate response surfaces
2:$\widehat{\mu}_1 = NN_2\left(Y^1 \sim \mathbf{X}^1\right)$	
3:$\tilde{D}^1 = Y^1 - \widehat{\mu}_0\left(\mathbf{X}^1\right)$	▷ Compute imputed treatment effects
4:$\tilde{D}^0 = \widehat{\mu}_1\left(\mathbf{X}^0\right) - Y^0$	
5:$\widehat{\tau}_1 = NN_3\left(\tilde{D}^1 \sim \mathbf{X}^1\right)$	▷ Estimate group-specific CATEs
6: $\widehat{\tau}_0 = NN_4\left(\tilde{D}^0 \sim \mathbf{X}^0\right)$	
7: $\hat{\tau}(\mathbf{x}) = g(\mathbf{x})\,\hat{\tau}_0(\mathbf{x}) + (1 - g(\mathbf{x}))\,\hat{\tau}_1(\mathbf{x})$	▷ Average estimates

Next, consider Nie and Wager (2021)'s R-learner model that uses a specific loss function to capture treatment effect heterogeneity. Minimizing this loss function is equivalent to fitting a weighted pseudo-outcome regression. The R-learner starts with estimating the covariate-specific propensity function $\pi(\mathbf{X}_i) = Pr(W_i = 1|\mathbf{X}_i)$ for $W_i = 1$, and the conditional mean of the outcome Y_i given the covariates $\mathbf{X}_i$: $m(\mathbf{X}_i) = E[Y|\mathbf{X}_i = x]$. The R-learner's CATEs are then obtained by minimizing

$$\widehat{\mathcal{L}}_R[\tau(.)] = \frac{1}{n}\sum_{i=1}^{n}(W_i - \widehat{\pi}(\mathbf{X}_i))^2\left[\frac{Y_i - \widehat{m}(\mathbf{X}_i)}{W_i - \widehat{\pi}(\mathbf{X}_i)} - \tau(\mathbf{X}_i)\right]^2 \tag{11}$$

$$= \frac{1}{n}\sum_{i=1}^{n}(W_i - \widehat{\pi}(\mathbf{X}_i))^2\left[\widehat{\psi}_R(\mathbf{X}_i) - \tau(\mathbf{X}_i)\right]^2 \tag{12}$$

Equation (11) implies that minimizing the loss function is equivalent to regressing the pseudo-outcome $\widehat{\psi}_R(\mathbf{X}_i)$ on the observed covariates weighted by $(W_i - \widehat{\pi}(\mathbf{X}_i))^2$. The pseudo-outcome is motivated by a semiparametric linear model that uses residuals from the regression of Y_i on X_i (i.e., $Y_i - m(\mathbf{X}_i)$) and the residuals from the regression of W_i on $\mathbf{X}_i$ to address potential confounding from $\mathbf{X}_i$.

The R-learner's CATEs are estimated via the weighted pseudo-outcome regression as follows. First, after splitting the data into k-folds, fit the nuisance functions $\widehat{m}(.)$ and $\widehat{\pi}(.)$ on the portion of the excluded data by minimizing the prediction errors via cross-validation. Second, plug in the estimates from the previous step to estimate $\hat{\tau}(\mathbf{x_i})$ by minimizing (11) via parameter tuning on the k-folds. The weighted pseudo-outcome regression can be fit by deep neural networks (or weighted ensemble learning) that allow modification of the loss function by passing the weights $(W_i - \widehat{\pi}(\mathbf{X}_i))^2$. The pseudocode for obtaining CATEs from the deep neural network-estimated R-learner is:

Algorithm: DNN-based R-learner	
Input: X,Y,W; **Output:** $\widehat{\tau}$	
1:$\widehat{m} = NN_1(Y \sim \mathbf{X})$	▷ Estimate *nuisance* parameters
2:$\widehat{\pi} = NN_2(W \sim \mathbf{X})$	
3:$\widehat{\psi}_R(\mathbf{X}_i) = \frac{Y_i - \widehat{m}(\mathbf{X}_i)}{(W_i - \widehat{\pi}(\mathbf{X}_i)}$	▷ Compute pseudo-outcome
4:$\hat{\tau} = NN_3(\widehat{\psi}_R \sim \mathbf{X}_i)$	▷ Estimate CATE

3.5 Estimating PATT from experiments with noncompliance

Scholars such as Ottoboni and Populos (2020) have developed the "PATT-C" model to obtain *population* average treatment effects on the treated (PATT) from experimental (e.g., RCTs) and observational samples in which noncompliance with treatment is prevalent. We focus here on presenting the PATT-C model. To this end, let Y_{iGR} be the potential outcome for individual i in group $G_i \in \{0,1\}$. $G = 0$ in Y_{iGR} when i is in the target population and $G = 1$ when i is in the experimental sample (e.g. RCT).

Define $W_i \in \{0,1\}$ as the binary treatment assignment. Let $R_i \in \{0,1\}$ denote whether or not i received the treatment. Because the treatment is randomly assigned in the experimental sample, W_i and R_i are observed in this sample when $G_i = 1$ for i. Let $X_i \in \{0,1\}$ be the pretreatment covariates that influence selection into the experimental sample, treatment assignment in the population, and treatment noncompliance. $W_i = 0$ for individuals in the population who do not receive the treatment. $W_i = 1$ for those who receive the treatment but can decide whether or not to accept the treatment.

Let $C_i \in \{0,1\}$. $C_i = 1$ when i complies with the received treatment, and $C_i = 0$ when i does not comply with the treatment. C_i is only observed for individuals in the treated group in the experimental sample. Hence, for individuals who comply with the treatment, $W_i = R_i$. For individuals in the population, we only observe R_i but not W_i. Five assumptions permit identification of PATT from experimental samples with treatment noncompliance (Ottoboni and Populos, 2020, 110). The first assumption is *consistency* which implies that each i has the same response to the received treatment whether or not i is in the experimental sample: $Y_{i0R} = Y_{i1R}$; $R = \{0,1\}$ $\forall i$. The second is *conditional independence* of compliance and sample and treatment assignment: $C_i \perp G_i, W_i | X_i, 0 < \mathbb{P}(C_i = 1|X_i) < 1$. The third assumption is *strong ignorability* of sample assignment for the *treated*: $(Y_{i01}, Y_{i11}) \perp G_i|(X_i, W_i = 1, C_i = 1)$. The fourth assumption is *strong ignorability* of sample assignment for the *control*: $(Y_{i00}, Y_{i10}) \perp G_i|(X_i, W_i = 1, C_i = 1; 0 < \mathbb{P}(G_i = 1|X_i, W_i = 1, C_i = 1) < 1$.

The third and fourth assumptions imply strong ignorability of sample assignment for treated and control non-compliers since compliance—as per the second assumption—is also independent of sample and treatment assignment conditional on the covariates. These two assumptions also presuppose that the response surface is the same for compliers in the experimental sample and the target population. The fifth assumption is one-sided noncompliance, $\mathbb{P}(R_i|W_i = 0) = 0$ $\forall i$, which implies that individuals assigned to control are not allowed to receive the treatment.

The estimated PATT $\widehat{\tau}_P$ from the sample with treatment assignment is the average causal effect of taking up treatment assigned on individuals who received treatment in the population. This follows from the first two assumptions, which ensure that the potential outcomes do not differ based on sample assignment or receipt of treatment: $\widehat{\tau}_P = \mathbb{E}\left(Y_{i01} - Y_{i100}|G_i = 0, R_i = 1\right)$. Building on this and the five aforementioned assumptions permits identification of the PATT $\widehat{\tau}_P$ from samples with treatment noncompliance:

$$\widehat{\tau}_P = \mathbb{E}_{01}\left[\mathbb{E}(Y_{i11}|G_i = 1, R_i = 1, X_i)\right] - \mathbb{E}_{01}\left[\mathbb{E}\left(Y_{i10}|G_i = 1, R_i = 0, C_i = 1, X_i\right)\right]$$

$\mathbb{E}_{01}\left[\mathbb{E}\left(.|..., X_i\right)\right]$ denotes the expectation with respect to the distribution of X_i for those in the target population that received the treatment (see (Ottoboni and Populos, 2020, 111)).

Two datasets are required to estimate the PATT $\widehat{\tau}_P$ from samples with treatment noncompliance. The first is the experimental or RCT sample which is the sample of individual participants drawn from the target population, but in which some participants have not complied with the treatment. The second is the commensurate observational data from the target population in which some individuals have received the treatment but may or may not have accepted the treatment. Estimating the PATT $\widehat{\tau}_P$ from these datasets with treatment noncompliance is implemented in four steps.

First, train a model via deep neural networks (or weighted ensemble learning) to predict the probability of compliance as a function of the covariates X_i by using the group assigned to treatment in the experimental sample. Second, use the model trained in the first step to predict which observations in the experimental sample assigned to the control group would have complied with the treatment had they been assigned to the treated group. Third, for both observed compliers in the treated group *and* predicted compliers in the control group in the experimental sample, train a model via deep neural networks (or weighted ensemble learning) using the covariates X_i and information about whether i received (R_i) the treatment to predict the outcome response in the sample. This third step leads to $E\left(Y_{i11}|G_i = 1; R_i, X_i\right)$ for $R_i \in \{0,1\}$. Fourth, estimate the potential outcome Y_{i1R} for all i that received the treatment in the target population by using the model from the third step. This produces the potential outcome for i in group G that receives the treatment (R). The difference between the mean counterfactual Y_{i11} and the mean counterfactual Y_{i10} leads to the estimate of $\widehat{\tau}_P$. The pseudo-code for estimating the PATT in settings with noncompliance via deep neural networks is summarized as:

Algorithm: DNN-estimated PATT

Input: X,*Y*,*W*, *R*; **Output:** $\widehat{\tau}_P$

1: $\widehat{C}_i = NN_1\left(C \sim (X, W)\right)$ ▷Train Models
2: $\widehat{Y}_{iGR} = NN_2(Y \sim (X, R))$
3: $E\left(Y_{i1R}|G_i = 1; R_i, X_i\right)$ for $R_i \in \{0,1\}$ ▷Predict outcome response using $X_i$and R_i
4: $\widehat{\tau}_P = E(Y_{i11}|G_i = 1; R_i = 1, X_i) - E\left(Y_{i10}|G_i = 1; R_i = 0, C_i = 1, X_i\right)$ ▷Estimate PATT

4 Deep learning estimation of treatment effects

This section provides an introduction for building deep neural network models for estimating the meta-learner CATEs and the PATT in settings with treatment noncompliance. Our package also provides functions to estimate the CATEs and the PATT via weighted ensemble learning (see Table 1).

Deep Neural Networks

We use deep learning based on deep neural networks to estimate CATEs from the meta-learner models and the PATT from datasets with treatment noncompliance. The deep neural networks method employed here for estimating CATEs and the PATT is broadly based on the architecture illustrated in Figure 1. This figure shows that deep neural networks are structured as stacks of input and output layers on top of each other with multiple hidden layers added between the input and output layers. More specifically, in the case of deep neural networks, the predictor variable and covariates from a given dataset are captured by the number of neurons that constitute the input layer of the neural network. The covariates that include the treatment variable thus comprise the input layer for the network. The deep neural network learning process begins with the forward propagation phase that takes in the raw data (neurons) from the input layer (see Figure 1) that can be trained to learn the potential outcome(s).

The neurons from the input layer then go through the hidden layers, where the neurons perform deep learning on the data. With respect to such learning, deep neural network estimation focuses on feed-forward networks tasked with minimizing the mean squared error in the prediction of observed outcomes. The information from this exercise is then passed to the next layer. This process is repeated until convergence is achieved or the iteration reaches the specified maximum number. After training, inputting the same unit into the networks of (for example) the meta-learner model of interest produces predictions to estimate the CATE for each unit, as shown in Figure 1.

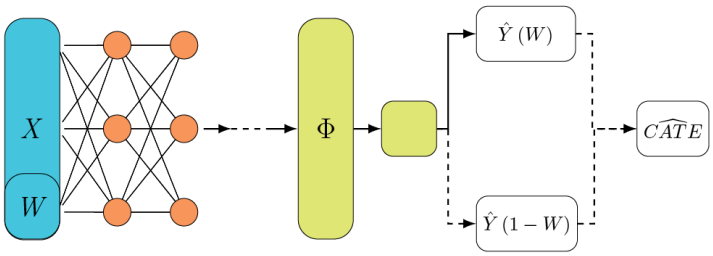

Figure 1: Estimating CATE Using Deep Neural Networks

4.1 Neural network architecture, model training, and treatment effects estimation

Deep neural networks (that is, deep learning) can be employed to estimate the CATEs and PATT from datasets by using Python libraries such as **Keras3** (Chollet et al., 2015) and **TensorFlow** (Abadi et al., 2015). The deep learning estimation procedure for causal inference consists of four steps. First, after loading and splitting their dataset into training and test sets, researchers need to build their neural network model for conducting the necessary estimation exercise. This entails defining their deep neural network architecture which involves specifying layers, activation functions, and the overall structure. Second, once the model is defined, researchers must compile it by specifying an optimizer for updating the model's weights to minimize the loss, define a loss function, and choose metrics.

While numerous optimization algorithms such as Adam, Stochastic Gradient Descent, AdaGrad, or RMSprop can be employed to update the model's weights for loss minimization, the applications presented below focus on using Adam (Adaptive Moment Estimation) for deep learning estimation of the CATE and PATT. Adam is an adaptive learning rate algorithm that leverages past gradient information to accelerate convergence, handle sparse gradients, dynamically adjust the learning rate for each individual parameter within a model, and minimize hyperparameter tuning. Apart from choosing an optimization algorithm, a loss function appropriate for the task must be defined, and metrics must be chosen to monitor during training and evaluation.

Third, after compiling the model, researchers need to train the model. This is implemented by fitting the model to the training data which involves setting the number of times the model iterates over the entire training dataset (denoted as `epochs`), choosing the number of samples processed before updating model weights (that is, `batch_size`), and setting the proportion of the training data used for validation during training via `validation_split`. Fourth, after training the model, researchers can proceed to estimate causal effects such as CATEs in the held-out test data. Since the loss function only minimizes the factual error to estimate $\hat{Y}$ (this is because we only observe one potential outcome for each unit), one has to artificially toggle the treatment to obtain $\hat{Y}(1)$ and $\hat{Y}(0)$ for each unit. After doing so, one can plug in the predictions to calculate the predicted CATEs for the meta-learner model of interest. To save space, we discuss the procedure and code for estimating the PATT in settings with

treatment noncompliance in Section 7 below.

5 Loading the DeepLearningCausal package and datasets

5.1 Functions, Reticulate, TensorFlow and Keras

Estimating the CATEs and PATT via deep neural networks requires the use of libraries in Python while also simultaneously using R code and modules. Our **DeepLearningCausal** package allows users to leverage Python libraries within the R programming environment and seamlessly use Python's robust deep learning ecosystem in their R session. To see how, users need to first install the **DeepLearningCausal** package from CRAN via `install.packages("DeepLearningCausal")`, and from its GitHub repository (hknd23/DeepLearningCausal). The **DeepLearningCausal** package automatically installs the **reticulate** R package that facilitates interoperability between Python and R, and it also automatically installs the **TensorFlow** and **Keras3** R packages as dependencies.

TensorFlow and **Keras3** allow for functions from the deep learning Python libraries **TensorFlow** and **Keras**(3.0) to work in tandem with R. However, given that the R packages only provide the interface, users must have the Python version of **TensorFlow** and **Keras** installed for the packages to work properly through reticulate. Hence, to streamline the installation process, users must call the function `python_ready()` to create a virtual environment for Python, and check for as well as install the necessary Python versions of the packages (TensorFlow, Keras3, NumPy). `python_ready()` thus ensures that both the R and Python versions of TensorFlow and Keras3 are installed. After installation of our package from CRAN, users can import the package by calling `library(DeepLearningCausal)` to import the functions in the package. Table 1 lists the functions in **DeepLearningCausal**:

Table 1: Brief description of functions in **DeepLearningCausal**

Function	Description
`metalearner_deeplearning()`	Deep neural network estimation of CATEs for S, T, X, and R-learner using reticulate, tensorflow and keras3
`pattc_deeplearning()`	Deep neural network estimation of PATT using reticulate, tensorflow and keras3
`conformal_plot()`	Conformal prediction for inference of ITEs from S and T-learner
`hte_plot()`	Heterogeneous Treatment Effects plot from meta-learner and PATT-C models
`metalearner_ensemble()`	Weighted ensemble learning estimation of CATEs for S, T, X, and R-learner using Super Learner (GitHub repo)
`metalearner_neural()`	Deep neural network estimation of CATEs for S, T, X, and R-learner using R neural net (GitHub repo)
`pattc_ensemble()`	Weighted ensemble learning estimation of PATT using Super Learner (GitHub repo)
`pattc_neural()`	Deep neural network estimation of PATT using R neural net (GitHub repo)

Our package first includes functions (see Table 1) which enable users to employ deep neural networks using **reticulate**, **TensorFlow** and **Keras3**, which gives them access to Python's deep learning libraries in their R session. Furthermore, our package includes the `hte_plot()` function that allows users to plot heterogeneous treatment effects and apply the conformal prediction (CP) procedure for generating predictive intervals for ITEs from two meta-learner models. All visualizations produced by the package are generated using **ggplot2** (Wickham, 2016).

The **DeepLearningCausal** package provides users with two additional options for estimating CATEs from the meta-learner models and the PATT in settings with treatment noncompliance. First, if users prefer to employ deep learning estimation without enabling Python, our package provides two functions that allow them to implement deep neural network estimation of the CATE and PATT by using the R neural net package: `metalearner_neural()` and `pattc_neural()`. Second, to employ the functions in our package to estimate the meta-learner CATEs and the PATT from the PATT-C model by using weighted ensemble learning, users need to install the **SuperLearner** package from CRAN:

5.2 Datasets

Two datasets are included as examples to illustrate the applicability of the functions in our package: the survey experiment sample (`exp_data_full`) and the commensurate representative population-

level survey response dataset (pop_data_full). These two datasets are described in the package's GitHub repository (hknd23/DeepLearningCausal). The survey experiment sample is generated from responses to survey questions fielded online to 728 respondents in India in 2022. After reading a vignette about a hypothetical crisis between their country and a foreign adversary, respondents are randomly assigned to the control group or the binary *strong leader* "treatment" variable coded as 1 for those exposed to a hawkish policy prescription for the said international crisis by a strong populist leader as opposed to a non-populist leader in their country.

```
data("exp_data_full")
str(exp_data_full)
```

```
#> 'data.frame':    514 obs. of  16 variables:
#>  $ strong_leader      : int  1 0 1 1 1 1 1 1 0 ...
#>  $ support_war        : int  0 1 0 1 1 0 1 1 1 1 ...
#>  $ compliance         : int  1 0 1 1 0 1 0 0 1 0 ...
#>  $ female             : num  1 0 1 1 1 0 1 0 1 1 ...
#>  $ age                : int  38 28 25 32 21 60 38 44 31 20 ...
#>  $ income             : int  7 2 5 7 9 7 4 3 4 2 ...
#>  $ religion           : int  1 6 1 9 1 6 5 6 6 9 ...
#>  $ hindu              : int  0 1 0 0 0 1 0 1 1 0 ...
#>  $ practicing_religion: int  4 1 1 1 4 3 2 1 1 1 ...
#>  $ education          : int  4 3 4 4 8 7 4 3 4 4 ...
#>  $ political_ideology : int  4 5 6 10 5 3 3 10 9 5 ...
#>  $ employment         : int  1 6 2 2 5 1 1 2 3 6 ...
#>  $ employed           : int  1 0 0 0 0 1 1 0 0 0 ...
#>  $ marital_status     : int  6 6 6 6 6 1 1 1 1 6 ...
#>  $ married            : int  0 0 0 0 0 1 1 1 1 0 ...
#>  $ job_loss           : int  1 1 4 1 1 1 4 4 2 1 ...
```

```
data("pop_data_full")
str(pop_data_full)
```

```
#> 'data.frame':    11813 obs. of  17 variables:
#>  $ strong_leader      : num  0 0 NA 0 0 NA 1 0 0 1 ...
#>  $ support_war        : int  NA NA NA NA NA NA NA NA NA NA ...
#>  $ compliance         : int  0 0 0 0 0 0 1 0 0 1 ...
#>  $ female             : num  0 1 0 1 1 0 1 0 0 0 ...
#>  $ age                : int  48 38 42 26 39 24 28 20 42 68 ...
#>  $ income             : int  3 6 3 2 4 3 4 4 4 3 ...
#>  $ religion           : num  31 31 31 31 31 31 31 31 31 31 ...
#>  $ hindu              : int  1 1 1 1 1 1 1 1 1 1 ...
#>  $ practicing_religion: int  2 2 4 2 2 2 3 4 4 2 ...
#>  $ education          : int  2 2 6 1 1 1 8 8 7 7 ...
#>  $ political_ideology : int  5 5 5 5 5 -1 5 5 5 5 ...
#>  $ employment         : int  3 6 1 2 6 3 6 7 1 5 ...
#>  $ employed           : int  1 0 1 1 0 1 0 0 1 0 ...
#>  $ marital_status     : int  1 1 1 1 1 6 1 6 1 1 ...
#>  $ married            : int  1 1 1 1 1 0 1 0 1 1 ...
#>  $ job_loss           : int  NA NA NA NA NA NA NA NA NA NA ...
#>  $ year               : int  2001 2001 2001 2001 2001 2001 2001 2001 2001 2001 ...
```

After random assignment to treatment or control, respondents are asked whether or not they support fighting a war against a foreign adversary. This generates the binary *support war* outcome measure. We record the vignette screen time latency and conduct factual manipulation checks to generate the binary *compliance* variable coded as 1 for respondents who understood and followed the instructions associated with the *strong leader* treatment and thus complied with this treatment; it is coded 0 for noncompliers. The survey experiment sample also includes confounders that operationalize the respondents' demographic features, dispositional traits, and attitudinal characteristics.

The second dataset in the package (pop_data_full) is the commensurate representative population-level World Values Survey response data from India for 1995, 2001, 2006, 2012, and 2022. This survey response data includes the binary *support war* outcome variable that mirrors the binary "support war" outcome measure in the India survey experiment sample. It also includes responses to a question about the respondents' preference for a *strong leader* that is compatible with the "strong leader"

treatment variable in the survey experiment sample. The binary *compliance* indicator in this data is 1 for those who responded to the strong leader preference question as this indicates compliance with the treatment proxy; it is 0 for respondents who did *not* answer this question as non-response indicates treatment noncompliance. The population-level survey response data include the same demographic, dispositional, and attitudinal covariates of respondents included in the survey experiment sample.

The following formula that specifies each model's (S-Learner, T-Learner, X-Learner, R-Learner, PATT-C) outcome measure *support war* and list of covariates drawn from the package's two example datasets helps us illustrate the applicability of the functions in **DeepLearningCausal**:

```
response_formula <- support_war ~ age + female + education + income + employed +
                                  job_loss + hindu + political_ideology
```

6 Deep learning estimation of meta-learner models

6.1 Deep neural networks for meta-learners

After initializing Python and setting up the virtual environment with `python_ready()`, the `metalearner_deeplearning()` function in **DeepLearningCausal** enables users to use TensorFlow and Keras3 to estimate CATEs from the S, T, X, and R-learner models. To save space, we focus on demonstrating below the applicability of the `metalearner_deeplearning()` function for the S-learner model by using the survey experiment dataset in our package. We also briefly present below some results from the deep learning-estimated T and X-learner models. More details about the procedure and results from the deep learning-estimated T, X and R-learner models by calling the `metalearner_deeplearning()` function are provided in the package's GitHub repository.

To begin with, the `metalearner_deeplearning()` function displayed below indicates that users can specify the arguments `train.data` and `test.data` to separately train the meta-learners on their training data and estimate CATEs with their test data. If a single dataset is specified, then the model will use cross-validation to train the meta-learners and estimate CATEs. Hence, for a single dataset, users can specify `nfolds`, which defines the number of folds to split data for cross-validation. Next, the `metalearner_deeplearning()` function permits users to specify the covariates in the S-learner model via `cov.formula = response_formula`. This includes the *support war* outcome measure and other confounders from the India survey experiment data, which is specified with `data = exp_data_full`. The treatment variable from this data is specified with `treat.var = "strong_leader"`. Users must specify `meta.learner.type = "S.Learner"` to estimate the S-learner model or specify `meta.learner.type=` to the meta-learner they want to estimate.

The arguments indicate that the `metalearner_deeplearning()` function permits users to substantially customize their deep neural network architecture for estimating CATEs from the meta-learner models. For instance, the `hidden.layer` argument permits users to specify the number of hidden layers and the number of neurons in each layer, while `hidden.activation` is the activation function for the hidden layers. Users can either specify a single value to use one activation function for all hidden layers (`hidden_activation = "relu"`) or a vector to set different activation functions for each hidden layer (`hidden_activation = c("relu", "tanh")`). Users can also specify arguments in `metalearner_deeplearning()` based on the outcome variable. For a binary outcome which involves a classification model, the output layer's activation function is `output_activation` while `output_units` is the number of units for the output layer. For regression models, the output layer's activation function is `output_activation = "linear"`. The argument `loss = "binary_crossentropy"` allows users to define the loss function for classification models, while `loss = "mean_squared_error"` defines the loss for regression models.

The arguments to reconfigure and train the deep neural networks for estimation include the optimization `algorithm` (options include "adam", "adagrad", "rmsprop", "sgd"), the number of `epochs` or iterations for a full pass of the data through the network to update weights, the size of each batch (`batch_size`) to split the training data, and `verbose` for monitoring the training progress. The arguments for metrics evaluation include `metrics = "accuracy"` for classification models and `metrics = "mean squared error"` for regression models. Arguments to specify the tuning of hyperparameters to monitor and mitigate overfitting include: `validation_split` that checks for overfitting, `patience` that stops the model early if overfitting is detected, and `dropout_rate` that mitigates overfitting to specify the proportion of neurons in each layer that can be dropped during training.

Next, the `plot(slearner_deep$m1_model_history[[1]])` in our **DeepCausalLearning** package allows users to illustrate the trace plot of the loss function and accuracy plots from the S-learner model. Moreover, as illustrated below, if the validation split is specified, then the trace and accuracy plots obtained from `plot(slearner_deep$m1_model_history[[1]])` will help users visualize how well the

deep neural network model is learning and identifying potential problems such as overfitting for both the training and validation datasets:

```
plot(slearner_deep$ml_model_history[[1]])
```

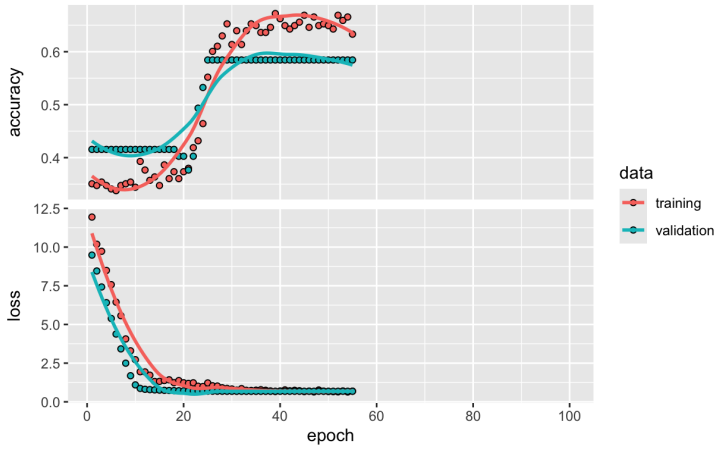

Figure 2: Accuracy Plot and Trace Plot of the Loss Function for the S-Learner

The bottom half of Figure 2 illustrates the trace plot of the loss function for the S-learner. This plot shows model performance through loss after each epoch. The y-axis of the plot shows the loss value (e.g., mean squared error, cross-entropy) while the x-axis indicates the training progress (e.g., epochs, iterations). The said trace plot figure reveals that the loss decreases steadily and smoothly on *both* the training and validation sets before flattening out. This means that the neural network model that we trained is learning effectively and generalizing well to new data.

The accuracy plot at the top half of Figure 2 shows how the model's accuracy changes over time (epochs or training steps) on the training dataset and a separate validation dataset. The x-axis of the accuracy plot represents the training epochs or iterations while the y-axis represents the accuracy score. Note that the gap between training and validation accuracy in the accuracy plot decreases, which indicates stable learning and an increase in accuracy. Overall, the steady decrease in loss but increase in accuracy illustrated in Figure 2 indicates stable learning. Furthermore, the trace and accuracy plots were cut off at the 54th epoch, meaning that the model stopped early to avoid overfitting. Finally, in addition to the S-learner, the function `metalearner_deeplearning()` also allows users to build and customize the deep neural network architecture for estimating the CATEs from the T, X, and R-learner models, which is described in the GitHub repository of our package.

6.2 CATEs, Conformal Inference, Heterogeneous Treatment Effects

Users can view the results of their deep neural network-estimated S and T-learner with `print(slearner_deep)` and `print(tlearner_deep)` respectively:

```
#> Method:
#> Deep Learning  S.Learner
#> Formula:
#> support_war ~ age + female + education + income + employed +    job_loss + hindu + political_ideology
#> Treatment Variable:  strong_leader
#> CATEs percentiles:
#>           10%           25%           50%           75%           90%
#> -0.0017127156 -0.0017046928 -0.0015885234 -0.0009122491 -0.0009065032

#> Method:
#> Deep Learning  T.Learner
#> Formula:
#> support_war ~ age + female + education + income + employed +    job_loss + hindu + political_ideology
#> Treatment Variable:  strong_leader
#> CATEs percentiles:
#>         10%         25%         50%         75%         90%
#> -0.24167932 -0.16082895 -0.04291850  0.02078736  0.05503685
```

The reported results provide information about the estimated CATE percentiles from the meta-learner model of interest. Next, the `conformal_plot()` function in **DeepLearningCausal** implements the conformal prediction framework to conduct direct inference of individual treatment effects (ITEs) obtained from the deep learning-estimated S and T-learner models. To see how, first note that individual treatment effects (ITEs) from the S-learner and T-learner are calculated as the difference between a subject's predicted outcome with and without a specific treatment.

The conformal intervals for the deep learning S and T-learner are generated directly within the `metalearner_deeplearning()` function when the argument `conformal = TRUE` is specified. This is displayed below for just the S-learner to save space:

```
slearner_deep <- metalearner_deeplearning(... , meta.learner.type = "S.Learner", ..., conformal = TRUE,
                                          alpha = 0.1, calib_frac = 0.5, prob_bound = TRUE,
                                             ... , seed = 1234)
```

Setting `conformal = TRUE` constructs uncertainty intervals around each estimated individual treatment effect (ITE). The argument `alpha = 0.1` specifies the miscoverage rate, corresponding to approximately 90% nominal coverage. `calib_frac = 0.5` divides the test sample into two parts: one is used for model prediction, and the other serves as a calibration set to compute residuals that quantify the typical deviation between model predictions and observed outcomes in finite samples.

The conformal adjustment is then determined by the *weighted quantile* of these residuals, where the weights are derived from the estimated propensity scores to correct for treatment–control imbalance. Setting `prob_bound = TRUE` ensures that the predicted probabilities and resulting conformal intervals are restricted to the [-1, 1] feasible range for binary outcomes. After estimation, the function stores the conformal intervals in `conformal` (S-learner) and `conformal` (T-learner) which can be visualized using:

```
conformal_plot(slearner_deep, binary.outcome = TRUE, prop = .2, seed = 1234)
conformal_plot(tlearner_deep, binary.outcome = TRUE, prop = .2, seed = 1234)
```

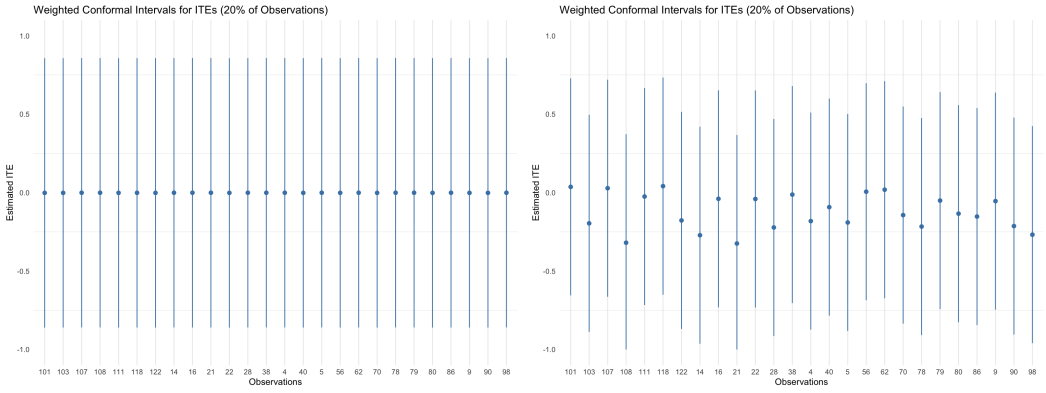

Figure 3: Conformal Predictions from Deep Neural Network-Based Meta Learners

The argument specifies that the outcome variable is binary, which constrains the conformal intervals to the logical probability bounds $[-1, 1]$. indicates that 20% of the test observations are randomly selected for display in the plot to maintain readability while preserving representativeness of the overall distribution. Figure 3 illustrates these intervals for the S-learner and T-learner. Each vertical line in these figures represents an observation's uncertainty range, and the length of the line reflects the model's local calibration residuals–wider intervals indicating greater uncertainty in the predicted ITEs.

Unlike the S-learner, which estimates both potential outcomes within a single network, the T-learner fits separate models for the treated and control groups. As a result, its conformal intervals often appear narrower and more heterogeneous across observations, reflecting that uncertainty is estimated conditional on treatment status rather than pooled across all units. By contrast, the S-learner tends to yield more conservative coverage due to shared outcome modeling, while the T-learner captures finer treatment-specific variation in individual effects.

Apart from the S- and T-learner models, our package permits users to customize their deep neural network architecture for deep learning estimation of the X- and R-learner models using **reticulate**, **TensorFlow**, and **Keras3**. The procedure and results for the R-learner are presented in the package's GitHub repository. After the X-learner model is estimated via their customized deep neural network architecture, users can view the model's results with `print(xlearner_deep)`:

```
print(xlearner_deep)

#> Method:
#> Deep Learning  X.Learner
#> Formula:
#> support_war ~ age + female + education + income + employed +    job_loss + hindu + political_ideology
#> Treatment Variable:  strong_leader
#> CATEs percentiles:
#>       10%       25%       50%       75%       90%
#> 0.2017969 0.2017969 0.2017969 0.2017969 0.2017969
```

These results report the deep neural network-estimated CATE percentiles from the X-learner model. The conformal prediction framework described earlier can also be employed to compute and illustrate conformal prediction (CP) intervals for individual treatment effects (ITEs) estimated from the two pseudo-outcome meta-learner models: the X and R-learner. The procedure as well as code for extracting CP intervals for the estimated ITEs from pseudo-outcome meta-learners have been developed and described by Alaa et al. (2023) and are thus not summarized here to save space.

As such, the estimated CATEs from all the meta-learner models in our package are stored in the element `CATEs`. The DNN-estimated CATEs from the (i) X-learner stored in `CATEs` can be called with `xlearner_deep$CATEs`, (ii) S-learner can be called with `slearner_deep$CATEs`, and (iii) T-learner can be called with `tlearner_deep$CATEs`. Users can thus employ the stored CATEs from all these three DNN-estimated meta-learner models (and the R-learner model) to illustrate and analyze treatment effect heterogeneity across distinct subgroups in their sample by using the `hte_plot()` function in our package. As displayed below, the `hte_plot()` function gives users the flexibility to enter any number of covariates for which she or he would like to assess heterogeneous treatment effects.

```
hte_plot(slearner_deep, selected_vars = c( "employed", "female", "political_ideology"),
         cut_points = c(.5,.5,5),  custom_labels= c( "Employed", "Unemployed",
                      "Male", "Female", "Centrist", "Right-wing Partisan"))
```

By default, the cutoff point in `hte_plot()` is set as the median of the interval and continuous covariates, and classifies binary variables (e.g., gender) into two categories (female and male). Using `hte_plot()`, we illustrate below the heterogeneous treatment effects (HTEs) from the estimated S, T, and X-learner model for the following subgroups of respondents in our experimental sample: male and female, employed and unemployed, and left-leaning versus right-leaning political ideology.

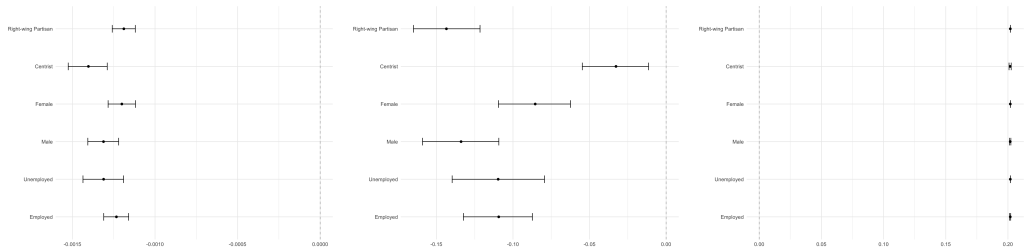

Figure 4: HTEs from Deep Neural Network-Based Meta Learners

The HTE estimates show some evidence of treatment effect heterogeneity obtained from the deep neural network-based S and X-learner but not the T-learner model. We also extract and illustrate the HTE plots obtained from the deep neural network estimated R-learner model in the GitHub repository of our package.

6.3 Comparing Meta-Learner Treatment Effects and Supplementary Analysis

The **DeepLearningCausal** package also enables users to plot the histograms and pairwise correlations of the estimated individual treatment effects from the deep neural network-estimated meta-learner models by using the **psych** package's `pairs.panels()`:

```
allCATEs_deep <- data.frame("S_learner" = slearner_deep$CATEs,
                            "T_learner" = tlearner_deep$CATEs,
                            "X_learner" = xlearner_deep$CATEs,
                            "R_learner" = rlearner_deep$CATEs)
psych::pairs.panels(allCATEs_deep, breaks = 30)
```

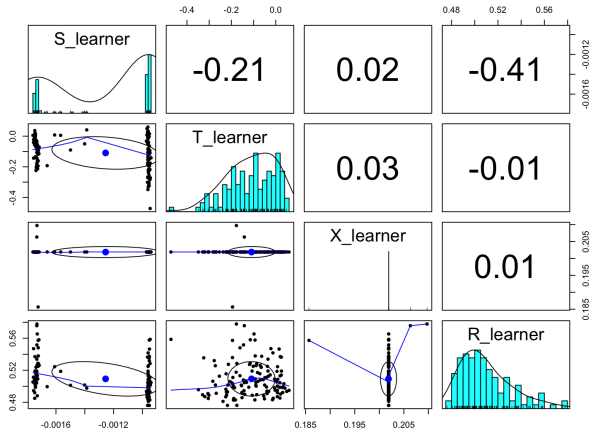

Figure 5: Treatment Effects Distribution and Correlations: Deep Neural Network-based Meta Learners

These figures suggest that the correlation of the estimated individual treatment effects across the meta-learner models estimated via deep neural networks using reticulate, TensorFlow, and Keras3 is weak and inconsistent. The individual treatment effects obtained from the deep neural network-based T-learner are also negatively correlated with those extracted from the S-learner and X-learner models.

To save space, we focus on reporting the meta-learner CATEs obtained from deep neural network estimation using reticulate, TensorFlow, and Keras3. However, the `metalearner_ensemble()` function (see Table 1) also enables users to estimate CATEs from the four meta-learner models via deep neural networks using R neural net. The function in our package permits users to employ weighted ensemble learning via the Super Learner approach for estimating CATEs from the S-, T-, X-, and R-learner. The procedure for estimating the meta-learner models using the two aforementioned functions is described in the package's GitHub repository (hknd23/DeepLearningCausal).

7 Deep learning for estimating PATT with treatment noncompliance

7.1 Deep neural networks for estimating the PATT

The `pattc_deeplearning()` function in the **DeepLearningCausal** package enables users to estimate the PATT by employing the survey experiment data and the population-level survey data in our package. To start with, the argument `exp.data = exp_data_full` in the aforementioned function specifies the data input for the experimental sample which serves as the training data. `pop.data = pop_data_full` is the data input for the representative population-level data (this is the World Values Survey in the package) which serves as the test data. `exp.data = exp_data_full` and `pop.data = pop_data_full` are both employed to estimate the PATT in settings with treatment noncompliance.

The argument `response.formula = response_formula` permits users to specify the outcome variable (e.g., *support war*) and the confounders in the PATT-C model. `treat.var` and `compl.var` allow users to specify the treatment variable and the compliance variable, respectively.

The arguments displayed above reveal that the `pattc_deeplearning()` function provides users with substantial options to customize and tune the hyperparameters of both the compliance and response models separately in the PATT-C estimator to obtain the PATT in settings with treatment noncompliance. These arguments include for example `exp.data`, `pop.data` and `treat.var` that users can employ to specify their data inputs. `compl.hidden.layer` permits users to specify the number of layers and neurons in each layer for the compliance model that users can specify, while `response.hidden.layer` is employed to specify the number of layers and neurons in each layer for the response model. The argument `compl.hidden.activation` is the activation function for the hidden layers (e.g., "relu") in the compliance model for each hidden layer, and `response.hidden.activation` denotes the activation function for the hidden layers in the response model.

Users can also specify the arguments for the response model based on the outcome variable. For a binary outcome variable, the activation function of the response model's output layer is `response.output_activation` which is set to "sigmoid". The argument `response.output_units` is the number of units for the response model's output layer. Next, users can define the response model's loss function as `response.loss = "binary_crossentropy"` for classification and `response.loss = "mean_squared_error"` for regression. The argument `compl.algorithm` and `response.algorithm` de-

note the optimization algorithms (e.g., "adam", "adagrad", "rmsprop") for the compliance and response models, respectively.

`compl.epoch` and `response.epoch` denote iterations of the full pass of the data through the network to update weights for the compliance and response models, respectively. The argument `batch_size` permits users to split the training data, while `verbose` allows users to monitor the training progress. The arguments for the evaluation metrics are `response.metrics = "accuracy"` for when the response model is a classification model and `response.metrics = "mean_squared_error"` for a regression model. Arguments that specify the tuning of hyperparameters to monitor and mitigate overfitting for deep neural networks when estimating the PATT are `compl.validation_split` for the compliance model, and `response.validation_split` for the response model. Users can specify `compl.patience` and `response.patience` to stop the model early if overfitting is detected for the compliance and response models. `compl.dropout_rate` permits users to set the proportion of neurons in each layer that can be dropped during training of the compliance model to mitigate overfitting. The argument `nboot = 1000` sets the number of bootstrap samples to 1,000. Finally, `plot(deeppattc$complier_history)` and `plot(deeppattc$response_history)` can be employed to illustrate the trace plots of the loss and metric functions of the complier and response models used for estimating the PATT. This enables users to assess whether their deep neural network architecture is learning and identifying overfitting when estimating PATT in data with treatment noncompliance.

Once the deep neural networks are trained on the experimental data, users can call `deeppattc$pop_counterfactual` to extract the predicted probabilities of the outcome measure for population treatment compliers *if* they are all in the treated group (the counterfactual treated group) and *if* they are all in the control group (the counterfactual control group). They can then use `plot(deeppattc)` to illustrate the distribution of these predicted probabilities of the outcome measure for population compliers in the counterfactual treated group (blue histogram) and counterfactual control group (red histogram):

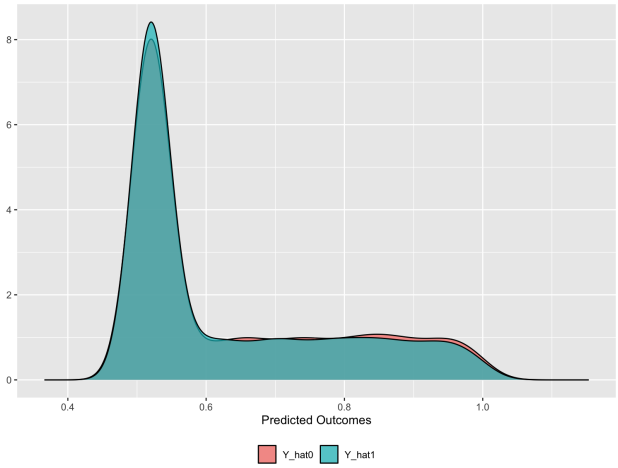

Figure 6: Counterfactual Predictions Deep Learning PATT-C

This figure indicates that the share of population compliers in the counterfactual treated group who support war is not different than their peers in the counterfactual control group. The predicted probabilities of the outcomes obtained from this exercise can also be employed for additional analyses, such as identifying members in the target population who are most receptive to the treatment.

7.2 Heterogeneous Treatment Effects and supplementary analysis

Our **DeepLearningCausal** package also enables users to extract and illustrate heterogeneous treatment effects for specified subgroups in the target population once the PATT has been estimated from datasets with treatment noncompliance using their deep neural network architecture. Heterogeneous treatment effects are estimated by taking differences across response surfaces for a given covariate, and response surfaces are estimated with neural network-based mean predictions when deep neural networks are employed for estimating the PATT. Users can call the `hte_plot()` function to extract and illustrate the heterogeneous treatment effects for the deep neural network-estimated PATT-C model. The HTE plot is shown below for the *support war* outcome measure for the subgroups listed earlier:

```
hte_plot(deeppattc, selected_vars = c( "employed", "female", "political_ideology"),
         cut_points = c(.5,.5,5),  custom_labels= c( "Employed", "Unemployed",
                                                     "Male", "Female", "Centrist",
```

```
"Right-wing Partisan"))
```

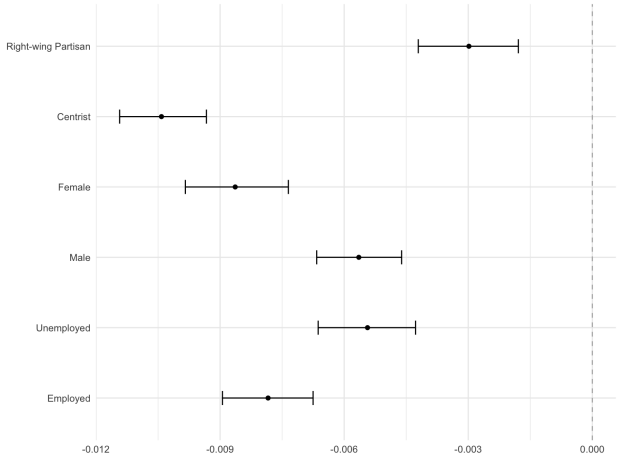

Figure 7: HTEs for Selected Covariates of Interest

In addition to deep learning estimation of the PATT in settings with treatment noncompliance, our **DeepLearningCausal** package also allows users to employ the R **neuralnet** package for estimating the PATT (from the PATT-C model) via deep neural networks with `pattc_neural()`. Additionally, the `pattc_ensemble()` function in **DeepLearningCausal** permits users to estimate the PATT from datasets that exhibit treatment noncompliance by using weighted ensemble learning from `library(SuperLearner)` that focuses on four candidate machine learning algorithms mentioned earlier. The procedure for estimating the PATT in settings with treatment noncompliance via deep neural networks (from R neuralnet) and weighted ensemble learning using the super learner is described in our package's GitHub repository (hknd23/DeepLearningCausal).

8 Conclusion

The rapid surge in research on causal machine learning has led to the development of statistical software packages that enable users to employ conventional machine learning, weighted ensemble learning or targeted learning to estimate a wide variety of causal models (e.g., van der Laan and Rose, 2011; Nie and Wager, 2021; Knaus, 2022; Hu and Ji, 2022; Zhao and Liu, 2023). More recent research on causal inference has, however, sought to demonstrate how deep causal learning, which leverages deep neural networks, can address challenges stemming from high-dimensional data, unobserved confounders, and selection bias (Goodfellow et al., 2016; Li et al., 2024; Koch et al., 2024). Yet barring some open-source Python code that introduces deep learning for causal inference (Koch et al., 2024), there is, to our knowledge, no R package that allows users to employ deep learning methods for estimating treatment effects in settings with noncompliance.

Our R package addresses this lacuna by offering functions that enable the use of deep learning—deep neural networks—to estimate the meta-learner CATEs in samples and the PATT from a newly developed causal estimand that addresses treatment noncompliance (Künzel et al., 2019; Ottoboni and Populos, 2020). Another feature of our package is that it enables users to import Python's deep learning modules directly into their R session to construct and customize their deep neural network architecture for estimating treatment effects. It also permits users to apply the conformal prediction (CP) procedure to generate predictive intervals for ITEs from a set of conditional mean regression meta-learner models.

Future work can build on the general deep neural network architecture for causal inference in numerous ways. For instance, R packages can be developed that allow users to employ specific deep neural network models such as Convolutional Neural Networks, Recurrent Neural Networks, Variational Autoencoders, and Conditional Generative Adversarial Networks. These specific models offer several benefits for causal inference such as learning non-linear relationships between features and counterfactual outcomes, imputation of missing counterfactual outcomes, learning latent representations of data, predicting the conditional outcome and propensity score, and extending causal inference to text data, networks, and images. The deep learning-estimated causal models in our R package can be applied to high-dimensional data, large-scale RCTs conducted for target populations, and observational data with unknown interventions and long-range temporal dependencies. Doing

so will expand the application of the causal models in the package across several fields that employ randomized trials, panel data, or high-dimensional observational data.

References

M. Abadi et al. Tensorflow: Large-scale machine learning on heterogeneous systems, 2015. URL https://www.tensorflow.org/. accessed 2025-11-05. [p104, 110]

A. Alaa and M. Van Der Schaar. Limits of estimating heterogeneous treatment effects: Guidelines for practical algorithm design. *Proceedings of the 35th International Conference on Machine Learning*, 80: 129–138, 2018. URL https://proceedings.mlr.press/v80/alaa18a.html. [p103, 106]

A. Alaa, Z. Ahmed, and M. van der Laan. Conformal meta-learners for predictive inference of individual treatment effects. *Proceedings of the 37th Conference on Neural Information Processing Systems*, 48:1–22, 2023. URL https://proceedings.neurips.cc/paper_files/paper/2023/file/94ab02a30b0e4a692a42ccd0b4c55399-Paper-Conference.pdf. [p106, 116]

J. J. Allaire and Y. Tang. *tensorflow: R Interface to 'TensorFlow'*, 2024. URL https://CRAN.R-project.org/package=tensorflow. R package version 2.16.0. [p104]

S. Athey and G. W. Imbens. Recursive partitioning for heterogeneous causal effects. *Proceedings of the National Academy of Sciences*, 113(27):7353–7360, 2016. URL https://doi.org/10.1073/pnas.1510489113. [p103]

P. Bach, M. S. Kurz, V. Chernozhukov, M. Spindler, and S. Klaassen. DoubleML: An object-oriented implementation of double machine learning in R. *Journal of Statistical Software*, 108(3):1–56, 2024. URL https://doi.org/10.18637/jss.v108.i03. [p104]

L. B. Balzer, M. L. Petersen, and M. J. van der Laan. Targeted estimation and inference for the sample average treatment effect in trials with and without pair-matching. *Statistics in Medicine*, 35(21): 3717–3732, 2016. URL https://doi.org/10.1002/sim.6965. [p103, 104]

K. Battocchi, E. Dillon, M. Hei, G. Lewis, P. Oka, M. Oprescu, and V. Syrgkanis. EconML: A Python Package for ML-Based Heterogeneous Treatment Effects Estimation, 2019. URL https://github.com/py-why/EconML. Version 0.x. [p104]

V. Chernozhukov, D. Chetverikov, M. Demirer, E. Duflo, C. Hansen, W. Newey, and J. Robins. Double/debiased machine learning for treatment and structural parameters. *The Econometrics Journal*, 21 (1):C1–C68, 2018. URL https://doi.org/10.1111/ectj.12097. [p104]

F. Chollet et al. Keras, 2015. URL https://keras.io. accessed 2025-11-05. [p104, 110]

D. Coffman and B. Griffin. *twangContinuous: Toolkit for Weighting and Analysis of Nonequivalent Groups - Continuous Exposures*, 2021. URL https://CRAN.R-project.org/package=twangContinuous. R package version 1.0.0. [p104]

I. Díaz, N. Williams, K. Hoffman, and E. Schneck. Non-parametric causal effects based on longitudinal modified treatment policies. *Journal of the American Statistical Association*, 2021. URL https://doi.org/10.1080/01621459.2021.1955691. [p104]

M. Farrell, T. Liang, and S. Misra. Deep neural networks for estimation and inference. *Econometrica*, 89 (1):181–213, 2021. URL https://doi.org/10.3982/ECTA16901. [p103]

A. N. Glynn and K. M. Quinn. An introduction to the augmented inverse propensity weighted estimator. *Political analysis*, 18(1):36–56, 2010. URL https://doi.org/10.1093/pan/mpp036. [p104]

I. Goodfellow, Y. Bengio, and A. Courville. *Deep Learning*. Cambridge, MA: MIT Press, 2016. URL https://mitpress.mit.edu/9780262035613/deep-learning/. [p104, 119]

E. Hartman, R. Grieve, R. Ramsahai, and J. S. Sekhon. From sate to patt: Combining experimental with observational studies to estimate population treatment effects. *Journal of Royal Statistical Society Series A. Stat. Soc*, 178(3):757–778, 2015. URL https://doi.org/10.1111/rssa.12094. [p103, 104]

J. Hill. Bayesian nonparametric modeling for causal inference. *Computational and Graphical Statistics*, 20(1):217–240, 2011. URL https://doi.org/10.1198/jcgs.2010.08162. [p103]

L. Hu and J. Ji. Cimtx: An R package for causal inference with multiple treatments using observational data. *R Journal*, 14(3):213–230, 2022. URL https://journal.r-project.org/articles/RJ-2022-058/. [p104, 119]

L. Hu, J. Ji, and F. Li. Estimating heterogeneous survival treatment effect in observational data using machine learning. *Statistics in Medicine*, 40(21):4691–4713, 2021. URL https://doi.org/10.1002/sim.9090. [p103]

G. W. Imbens and D. Rubin. *Causal inference for statistics, social, and biomedical sciences: An introduction*. Cambridge, NY: Cambridge University Press, 2015. URL https://doi.org/10.1017/CBO9781139025751. [p103, 105]

F. Johansson, U. Shalit, and D. Sontag. Learning representations for counterfactual inference. in. *Proceedings of the 35th International Conference on Machine Learning*, 48:3020–3029, 2016. URL https://dl.acm.org/doi/10.5555/3045390.3045708. [p104]

G. Jonzon, M. Sachs, and E. Gabriel. Accessible computation of tight symbolic bounds on causal effects using an intuitive graphical interface. *R Journal*, 15(4):53–68, 2023. URL https://journal.r-project.org/articles/RJ-2023-083/. [p104]

T. Kalinowski, J. J. Allaire, and F. Chollet. *keras3: R Interface to 'Keras'*, 2025. URL https://CRAN.R-project.org/package=keras3. R package version 1.4.0. [p104]

M. Knaus. Double machine learning-based program evaluation under unconfoundedness. *The Econometrics Journal*, 25(3):602–627, 2022. URL https://doi.org/10.1093/ectj/utac015. [p104, 119]

B. J. Koch, T. Sainburg, P. E. Geraldo, S. Jiang, Y. Sun, and J. G. Foster. A primer on deep learning for causal inference. *Sociological Methods and Research*, 54(2):1–51, 2024. URL https://doi.org/10.1177/00491241241234866. [p103, 104, 119]

S. Künzel, J. Sekhon, P. Bickel, and B. Yu. Metalearners for estimating heterogeneous treatment effects using machine learning. *Proceedings of the National Academy of Sciences*, 116(1):4156–4165, 2019. URL https://doi.org/10.1073/pnas.1804597116. [p103, 104, 105, 106, 107, 119]

L. Lei and E. Candès. Conformal inference of counterfactuals and individual treatment effects. *Journal of the Royal Statistical Society Series B: Statistical Methodology*, 83(5):911–938, 2021. URL https://doi.org/10.1111/rssb.12445. [p106]

Z. Li, X. Guo, and S. Qiang. A survey of deep causal models and their industrial applications. *Artificial Intelligence Review*, 57(1):298, 2024. URL https://doi.org/10.1007/s10462-024-10886-0. [p103, 119]

X. Nie and S. Wager. Quasi-oracle estimation of heterogeneous treatment effects. *Biometrika*, 108(2):299–319, 2021. URL https://doi.org/10.1093/biomet/asaa076. [p103, 104, 107, 108, 119]

G. Okasa. Meta-learners for estimation of causal effects: Finite sample cross-fit performance. *arXiv preprint arXiv:2201.12692*, 2022. URL https://doi.org/10.48550/arXiv.2201.12692. [p104, 105]

K. N. Ottoboni and J. V. Populos. Estimating population average treatment effects from experiments with noncompliance. *Journal of Causal Inference*, 8:108–130, 2020. URL https://doi.org/10.1515/jci-2018-0035. [p103, 104, 108, 109, 119]

J. Pearl. *Causality*. Cambridge, NY: Cambridge University Press, 2009. URL https://doi.org/10.1017/CBO9780511803161. [p103]

S. Powers, J. Qian, K. Jung, A. Schuler, N. Shah, T. Hastie, and R. Tibshirani. Some methods for heterogeneous treatment effect estimation in high dimensions. *Statistics in Medicine*, 37(11):671–679, 2018. URL https://doi.org/10.1002/sim.7623. [p103]

M. Ratkovic. *SVMMatch: Causal Effect Estimation and Diagnostics with Support Vector Machines*, 2015. URL https://CRAN.R-project.org/package=SVMMatch. R package version 1.1. [p104]

M. S. Schuler and S. Rose. Targeted maximum likelihood estimation for causal inference in observational studies. *American Journal of Epidemiology*, 185(1):65–73, 2019. URL https://doi.org/10.1093/aje/kww165. [p103, 104]

S. Tikka. Identifying counterfactual queries with the R package cfid. *R Journal*, 15(2):330–343, 2023. URL https://journal.r-project.org/articles/RJ-2023-053/. [p103, 104]

K. Ushey, J. J. Allaire, and Y. Tang. *reticulate: Interface to 'Python'*, 2025. URL https://CRAN.R-project.org/package=reticulate. R package version 1.42.0. [p104]

M. J. van der Laan and S. Rose. *Targeted Learning: Causal Inference for Observational and Experimental Data*. Springer, NY: Springer Nature, 2011. URL https://link.springer.com/book/10.1007/978-1-4419-9782-1. [p103, 104, 119]

A. Wager and S. Athey. Estimation and inference of heterogeneous treatment effects using random forests. *Journal of the American Statistical Association*, 113(523):1228–1242, 2019. URL https://doi.org/10.1080/01621459.2017.1319839. [p106]

H. Wickham. *ggplot2: Elegant Graphics for Data Analysis*. Springer-Verlag New York, 2016. URL https://ggplot2.tidyverse.org. [p111]

J. Xu, T. Shinkre, and J. Brand. *htetree: Causal Inference with Tree-Based Machine Learning Algorithms*, 2023. URL https://CRAN.R-project.org/package=htetree. R package version 0.1.18. [p104]

L. Yao, S. Li, Y. Li, M. Huai, J. Gao, and A. Zhang. Representation learning for treatment effect estimation from observational data. *Advances In Neural Information Processing Systems*, 31(1):2633–2643, 2018. URL https://doi.org/10.1002/sim.7623. [p103, 104]

Y. Zhang, A. Bellot, and M. Schaar. Learning overlapping representations for the estimation of individualized treatment effects. *Proceedings of the Twenty Third International Conference on Artificial Intelligence and Statistics*, pages 1005–1014, 2020. URL https://doi.org/10.48550/arXiv.2001.04754. [p104]

Z. Zhang, Z. Hu, and C. Liu. Estimating the population average treatment effect in observational studies with choice-based sampling. *The International Journal of Biostatistics*, 15(1):965–972, 2019. URL https://doi.org/10.1515/ijb-2018-0093. [p103]

Y. Zhao and Q. Liu. Causalml: Python package for causal inference machine learning. *SoftwareX*, 21: 101294, 2023. URL https://doi.org/10.1016/j.softx.2022.101294. [p104, 119]

Nguyen Khoi Huynh
Hitotsubashi Institute for Advanced StudyHitotsubashi University
Kunitachi, Tokyo, Japan
ORCID: 0000-0002-6234-7232
khoinguyen.huynh@r.hit-u.ac.jp

Yang Yang
Department of Political ScienceThe Pennsylvania State University
University Park, PA, USA
ORCID: 0009-0004-6135-4555
yky5272@psu.edu

Bumba Mukherjee
Department of Political ScienceThe Pennsylvania State University
University Park, PA, USA
ORCID: 0000-0002-3453-601X
bumba.mukherjee@psu.edu